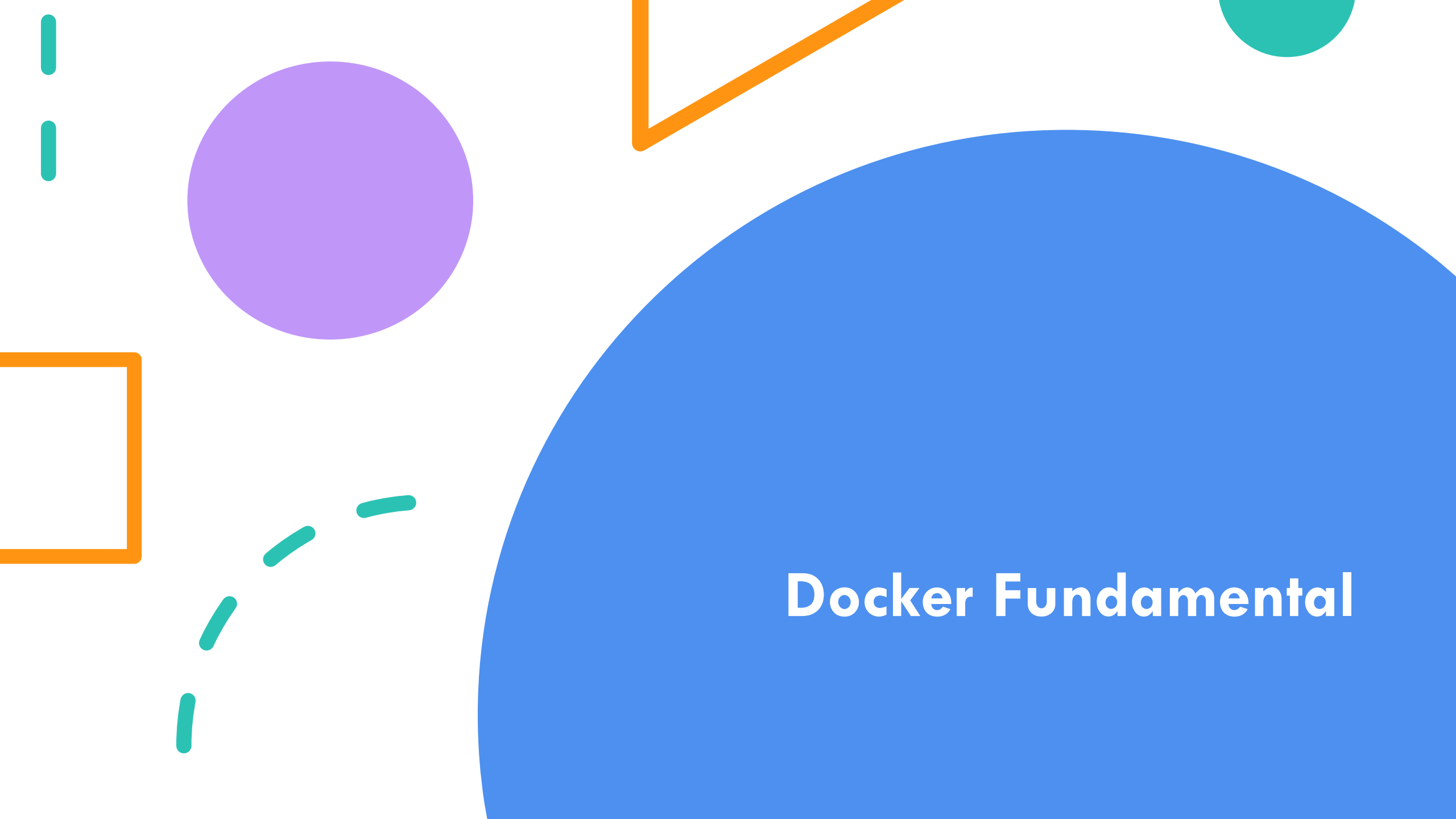




Docker Fundamental



Today's Lesson

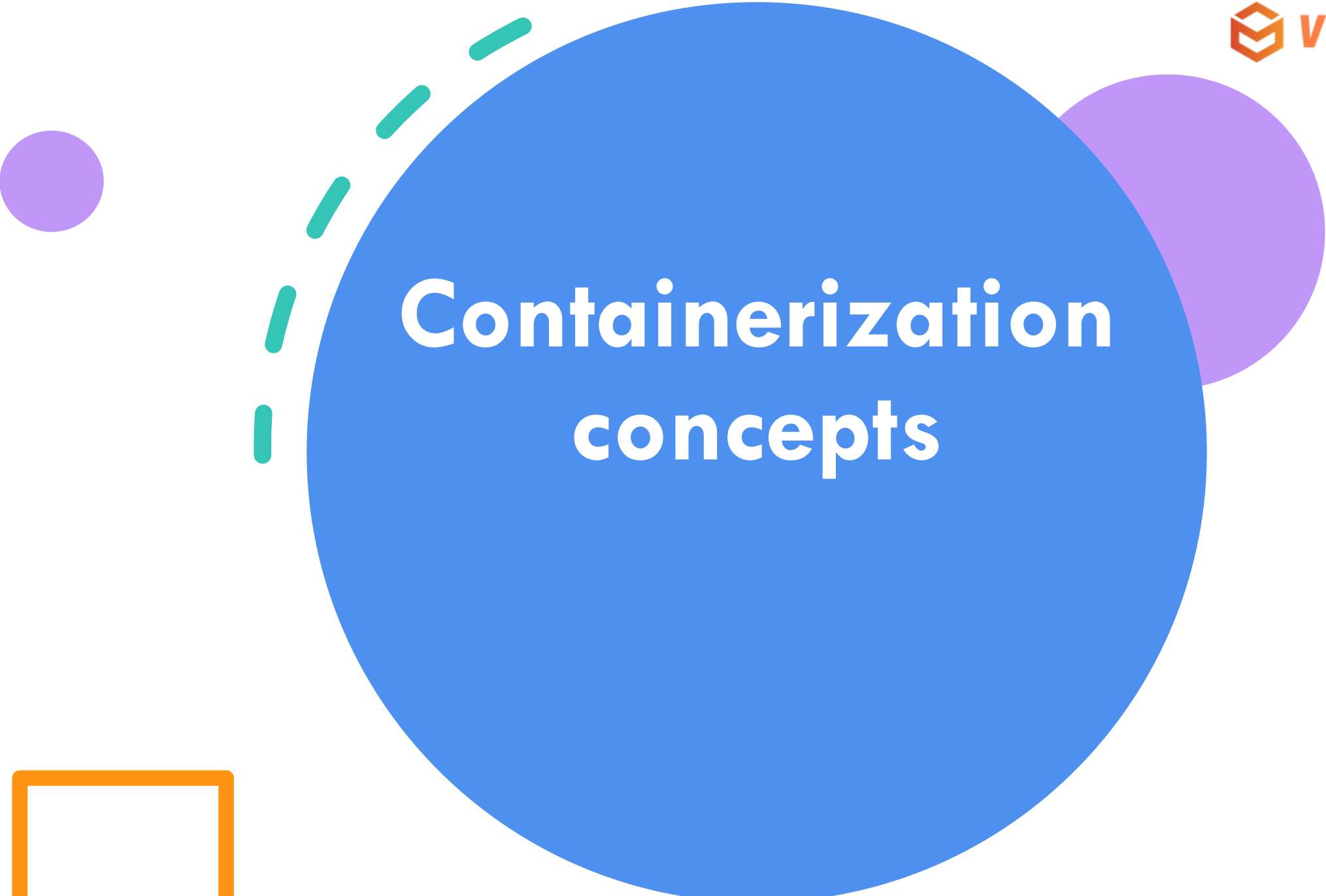
Containerization concepts

Docker architecture

Basic Docker commands

Image & Container

Container registry



Containerization concepts

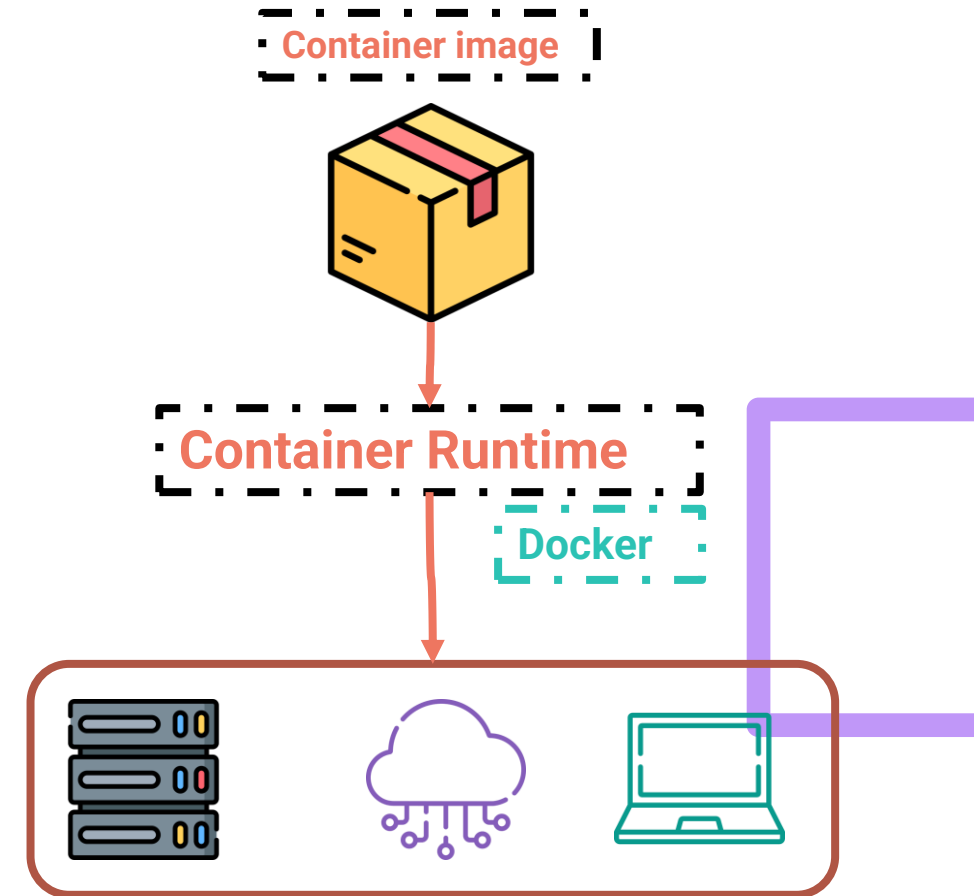
What is Containerization

- **Containerization** is a lightweight virtualization technology that packages applications and their dependencies into portable, self-contained units called **containers**. These containers can run consistently **across different computing environments**.
- In order for software to run, we need:
 - Source code
 - Host (server, virtual machine, cloud, personal laptop)
 - And dependencies for application to run



How **container** work?

- Container pack source code, and dependencies into **package**
- The **package** need to be run on **container runtime(docker)**
- And container runtime will be run on any Host (server, virtual machine, cloud, personal laptop) (Linux, Windows, MacOS)



Why container?

Portability

- "Write once, run anywhere" approach
- Consistent behavior across development, testing, and production

Efficiency

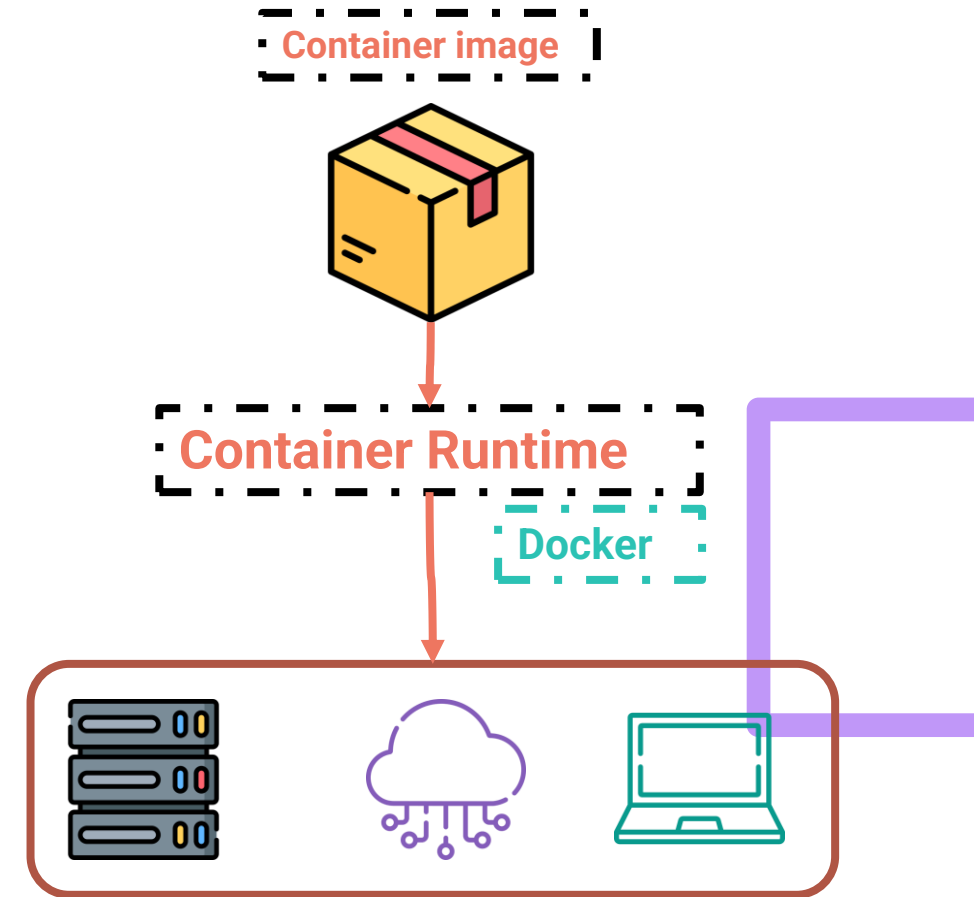
- Lightweight compared to virtual machines
- Faster startup times
- Better resource utilization

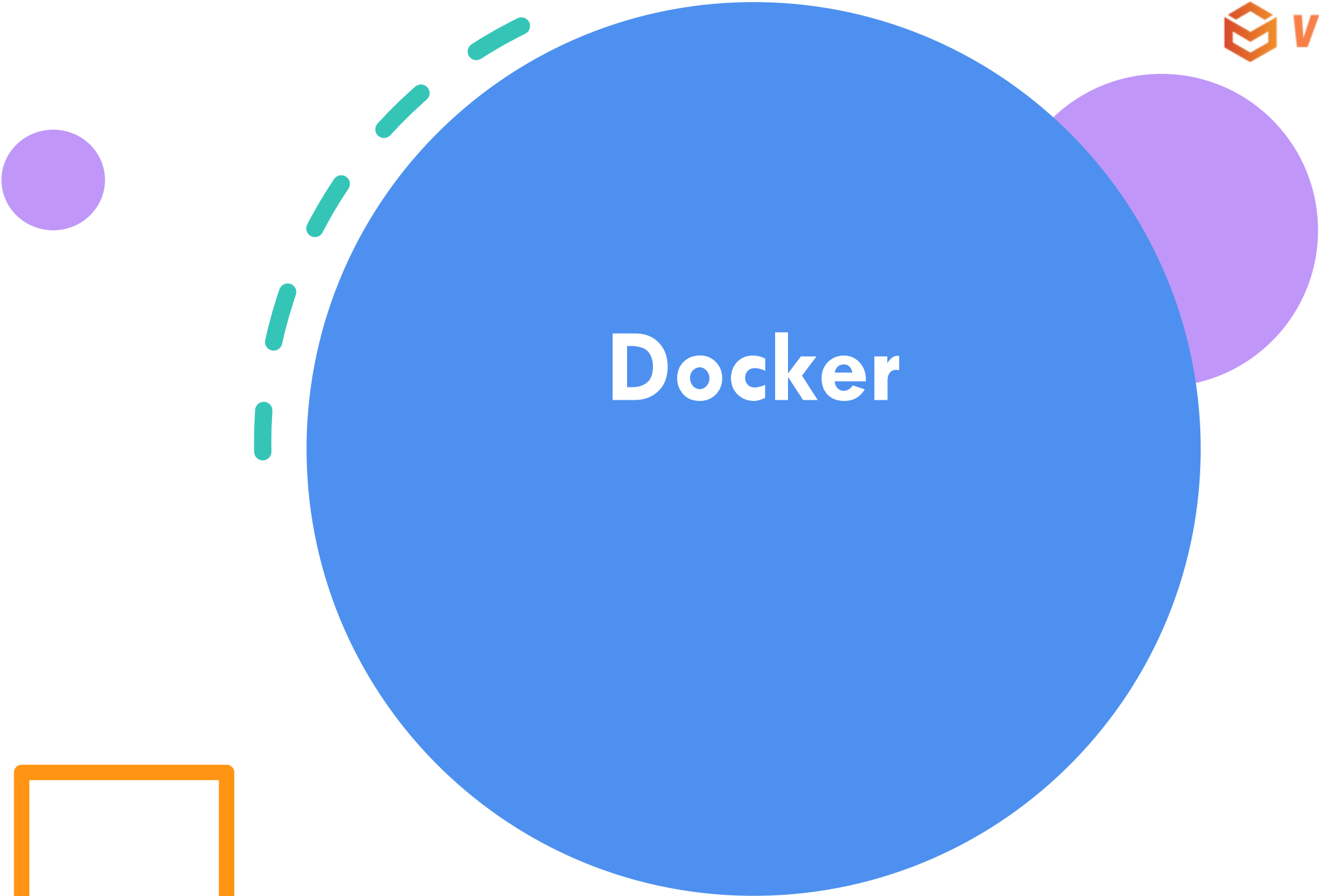
Isolation

- Applications run in separate environments
- Conflicts between dependencies are eliminated

Scalability

- Easy to scale applications
- Supports microservices architecture

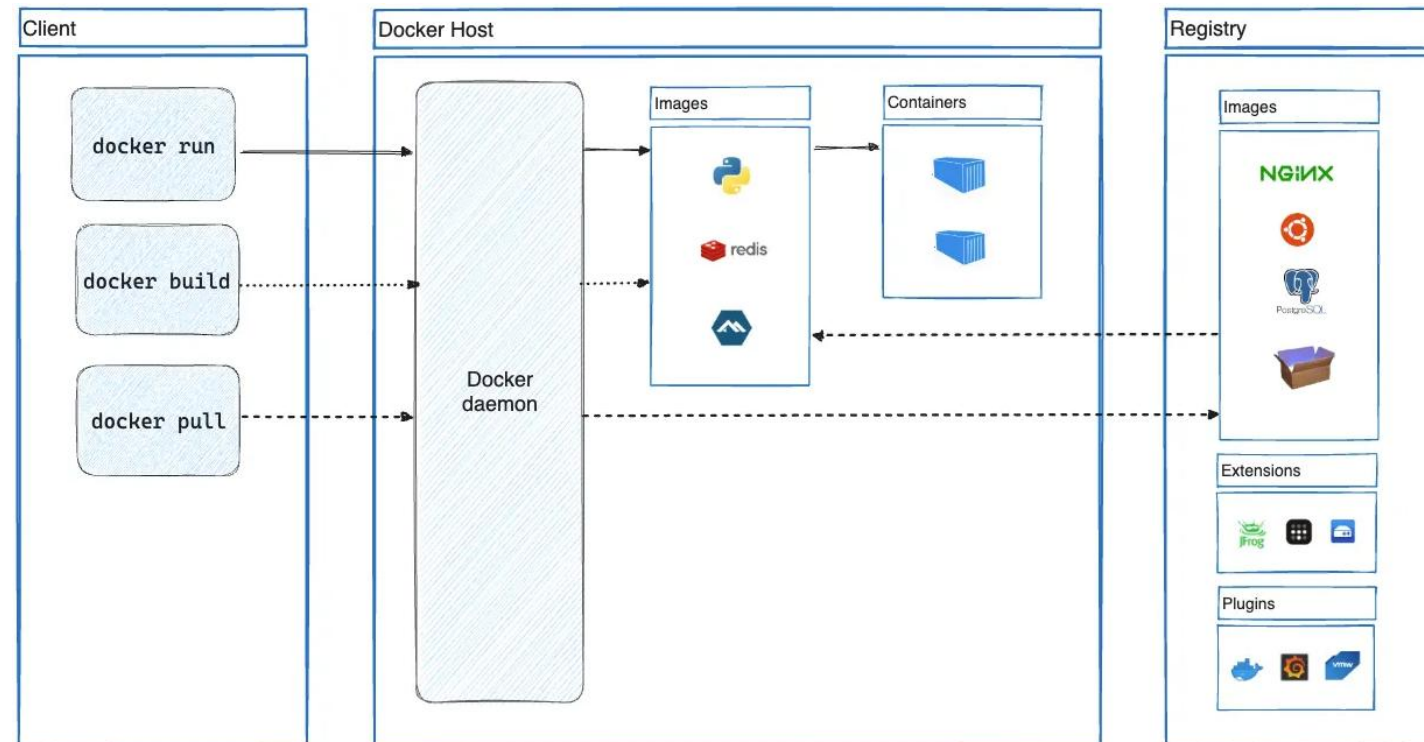




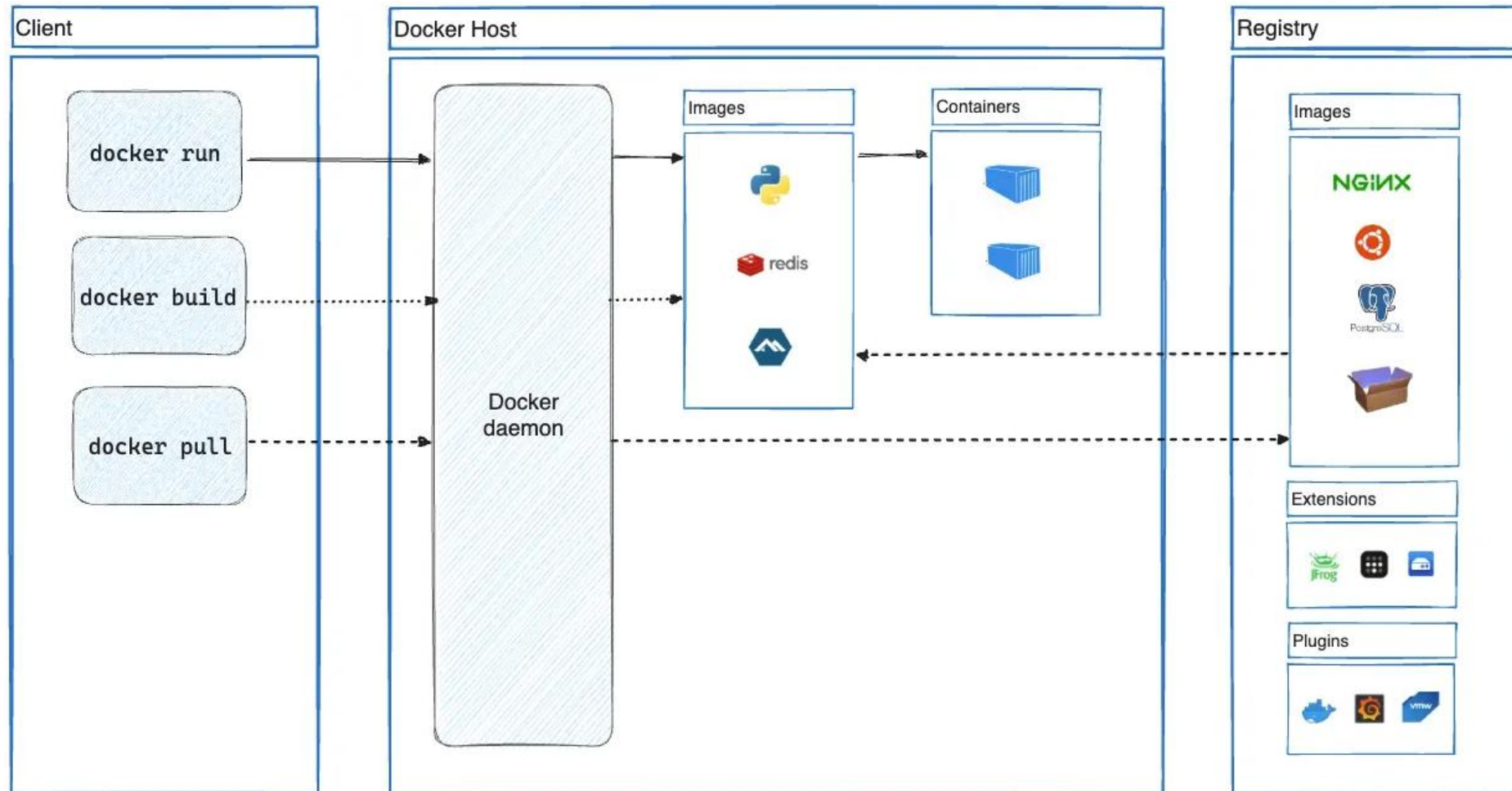
Docker

What is Docker?

- "Docker is an open platform for developing, shipping, and running applications." 
- "Docker is one of the **Container Runtime**, can install as **Deamon** in Linux (Windows) machine. Provide CLI to manage Container resource(image, network, volume ...) and container life cycle(stop, start,...)"



Docker Architecture



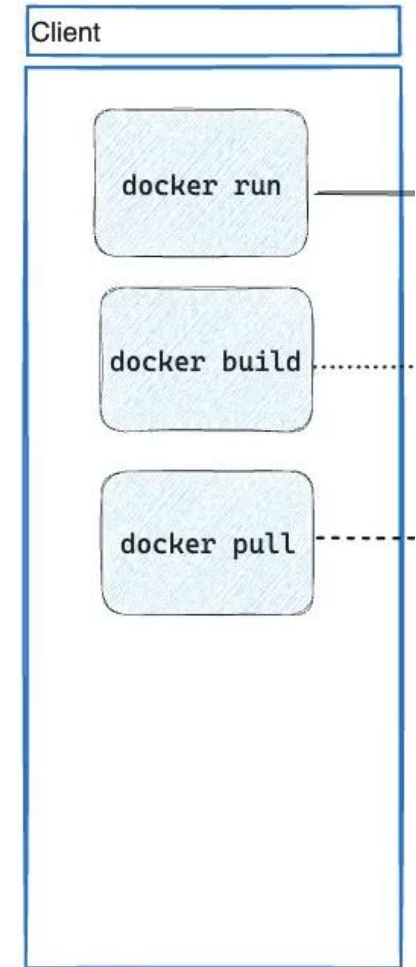
Docker Client

The Docker Client is the primary interface through which users interact with Docker. This is most commonly the Command Line Interface (CLI).

- It translates user commands like `docker ps` into REST API requests.
- These requests are sent to the Docker Daemon for processing.
- A single client can communicate with multiple daemons.

Basic Command

- `docker ps`: list all running container
- `docker pull <image>`: pull image from registry
- `docker run <image>`: run container from image



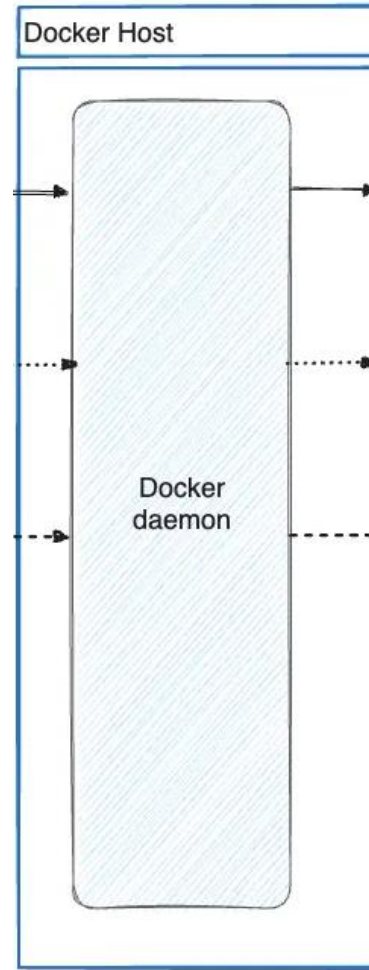
Docker Host & Deamon

Docker Host: This is the machine where the magic happens. It runs the Docker Daemon (dockerd) and provides the environment to execute and run containers.

The Docker Daemon (dockerd):

The **Docker Daemon** is the persistent background process that acts as the brain of your Docker installation.

- It runs on the **Docker Host**.
- It listens for API requests from the Docker Client.
- It manages all **Docker objects**: images, containers, networks, and volumes.
- It can communicate with other daemons to manage Docker services in a multi-host environment (like a Docker Swarm cluster).

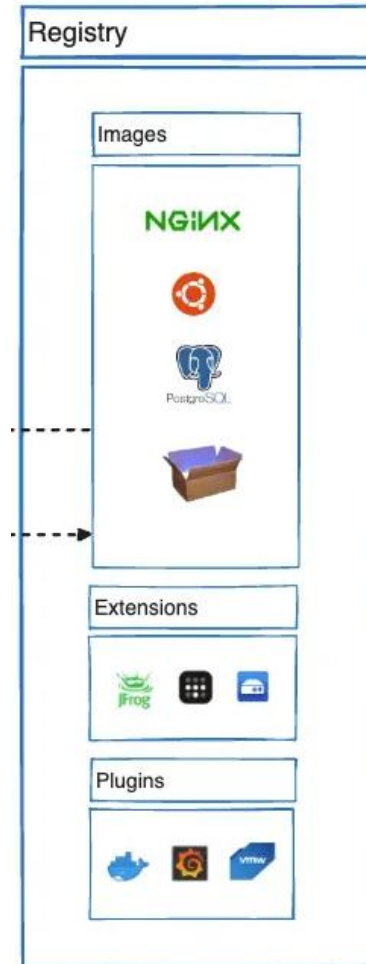


Docker Registry

- A Docker Registry is storage system for Docker images.

Image Commands:

- `docker pull <image_name>`: Downloads an image from a configured registry to your local Docker Host.
- `docker push <image_name>`: Uploads a local image to a registry.



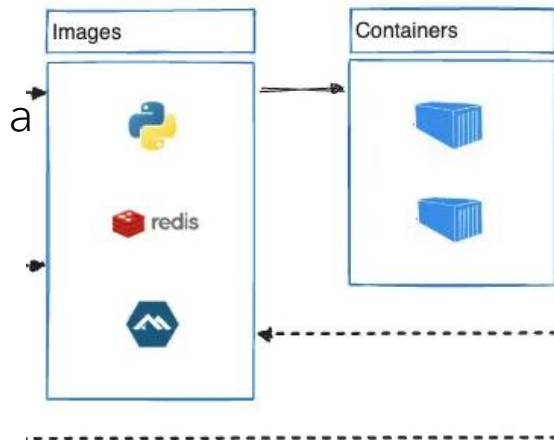
Docker Object

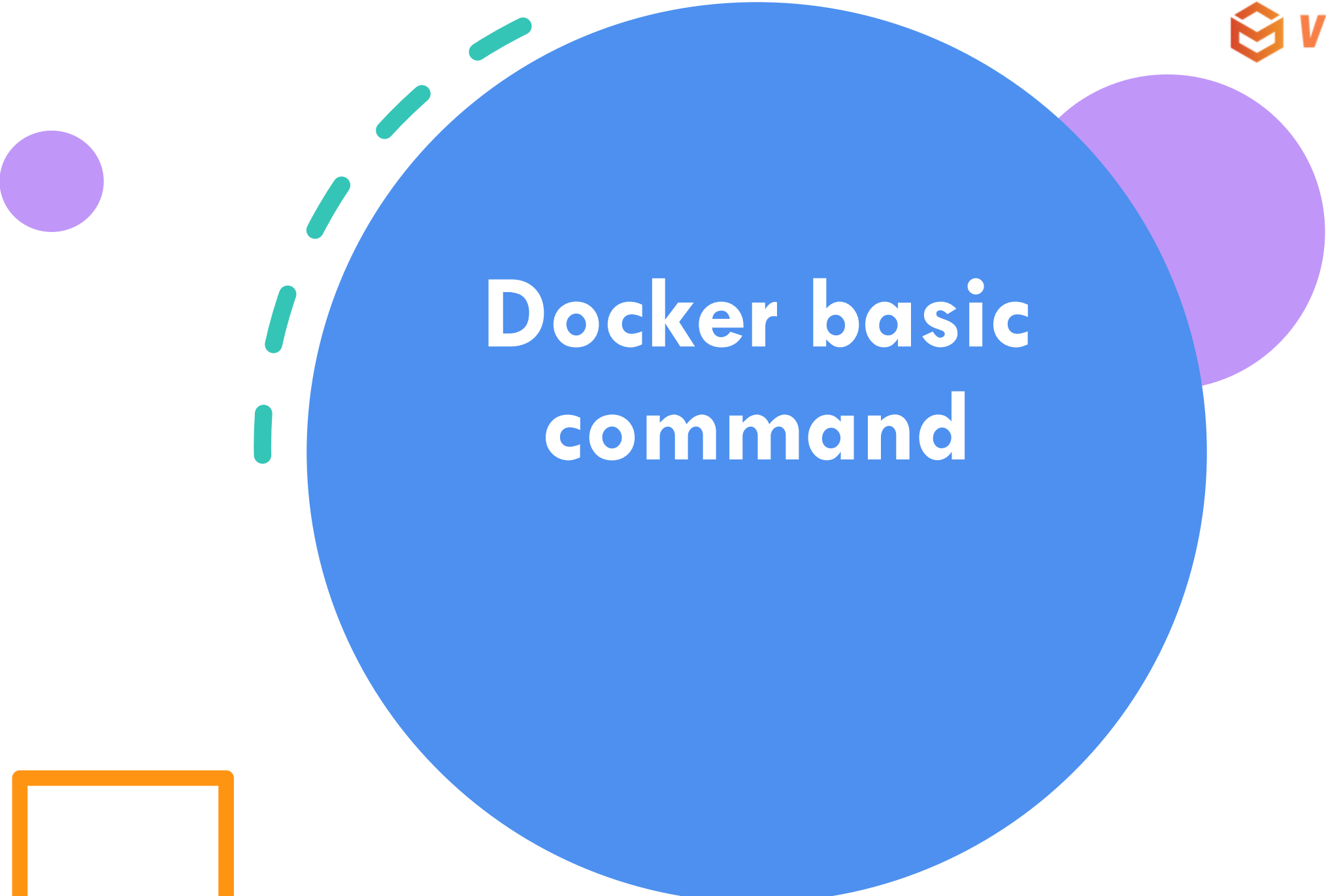
Images

- An image is a read-only, inert template that contains the instructions for creating a Docker container. Think of it as a blueprint or a class in object-oriented programming.

Container

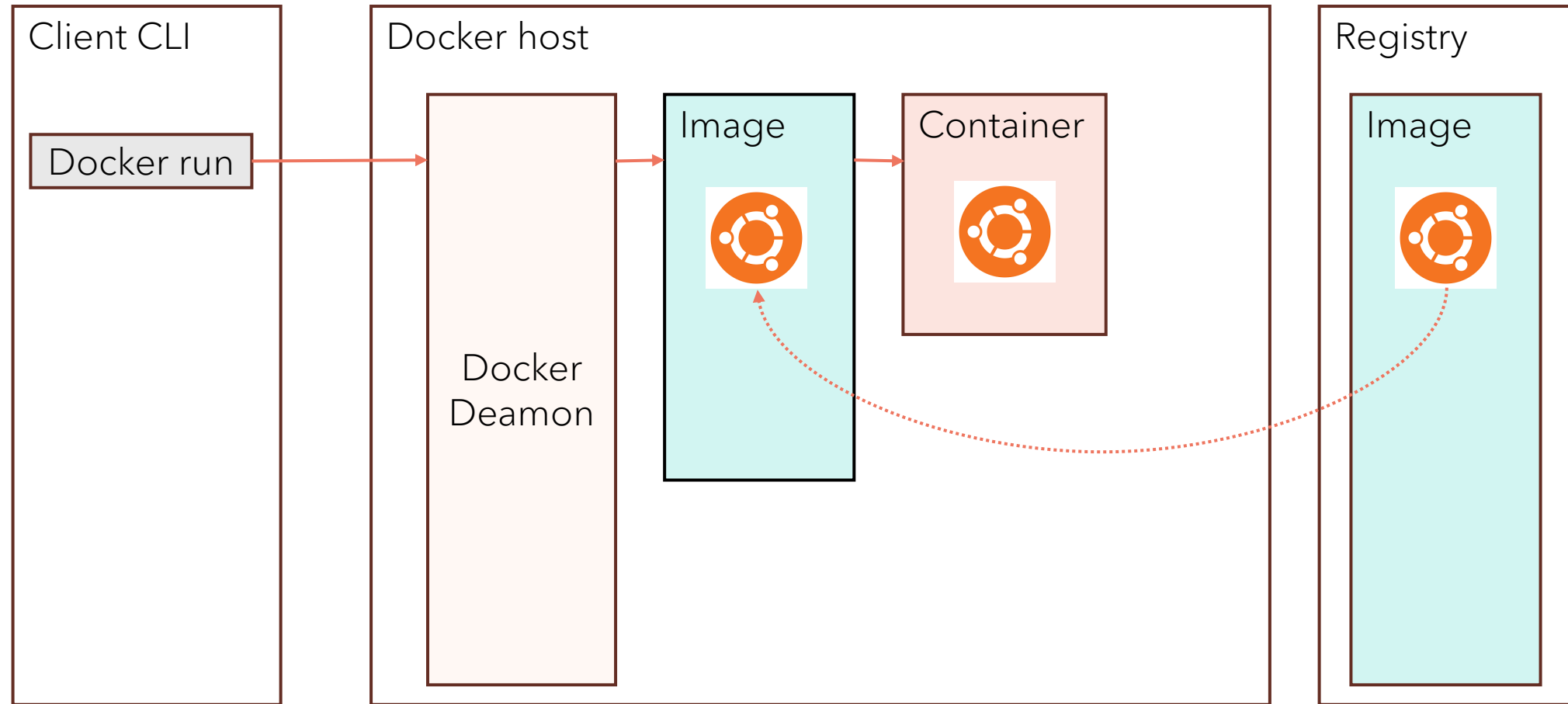
- A container is a runnable instance of an image. You can create, start, stop, move, or delete a container using the Docker API or CLI. You can connect a container to one or more networks, attach storage to it, or even create a new image based on its current state.
- *"Other object: Volume, Network (will be cover on later lesson)"*



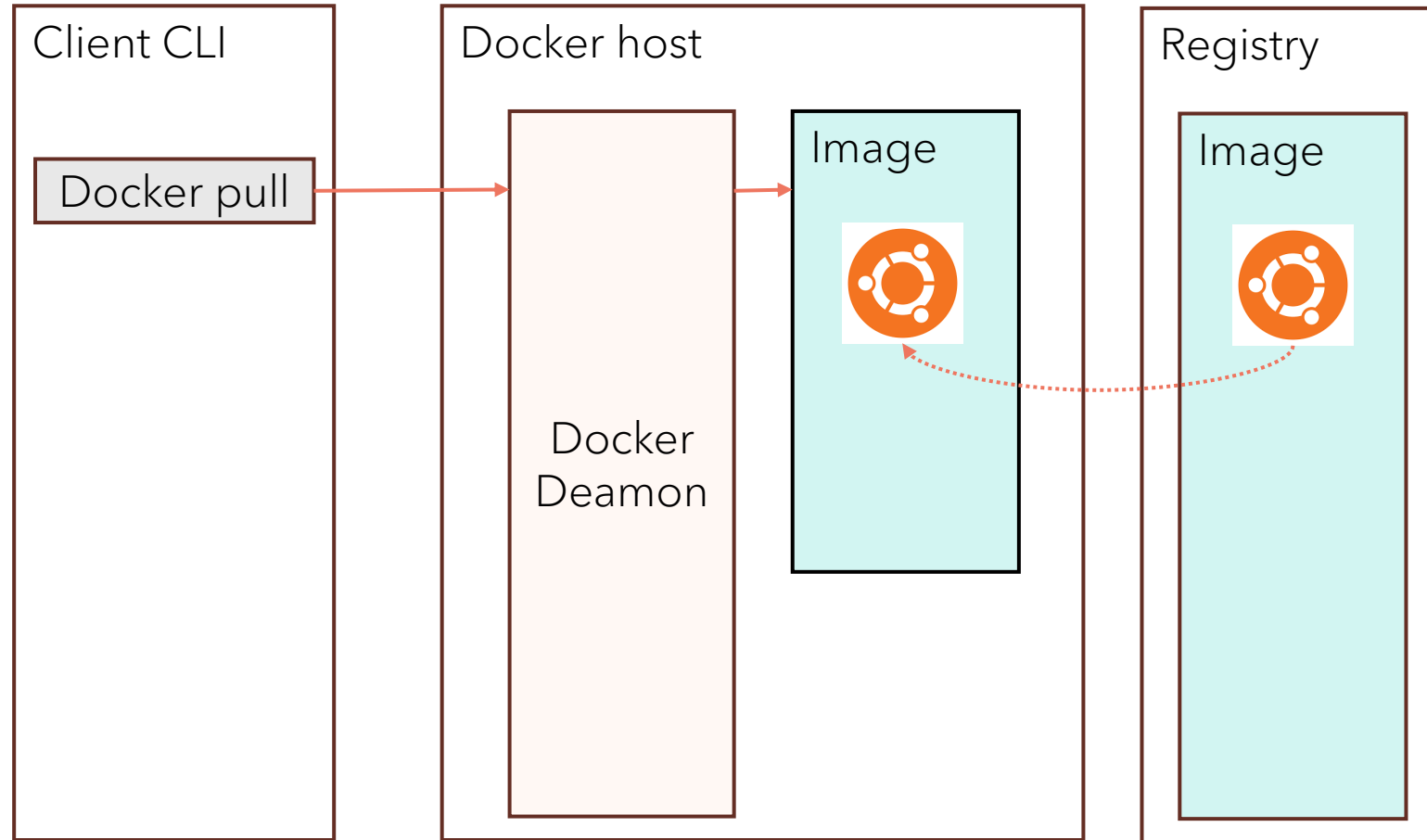


Docker basic command

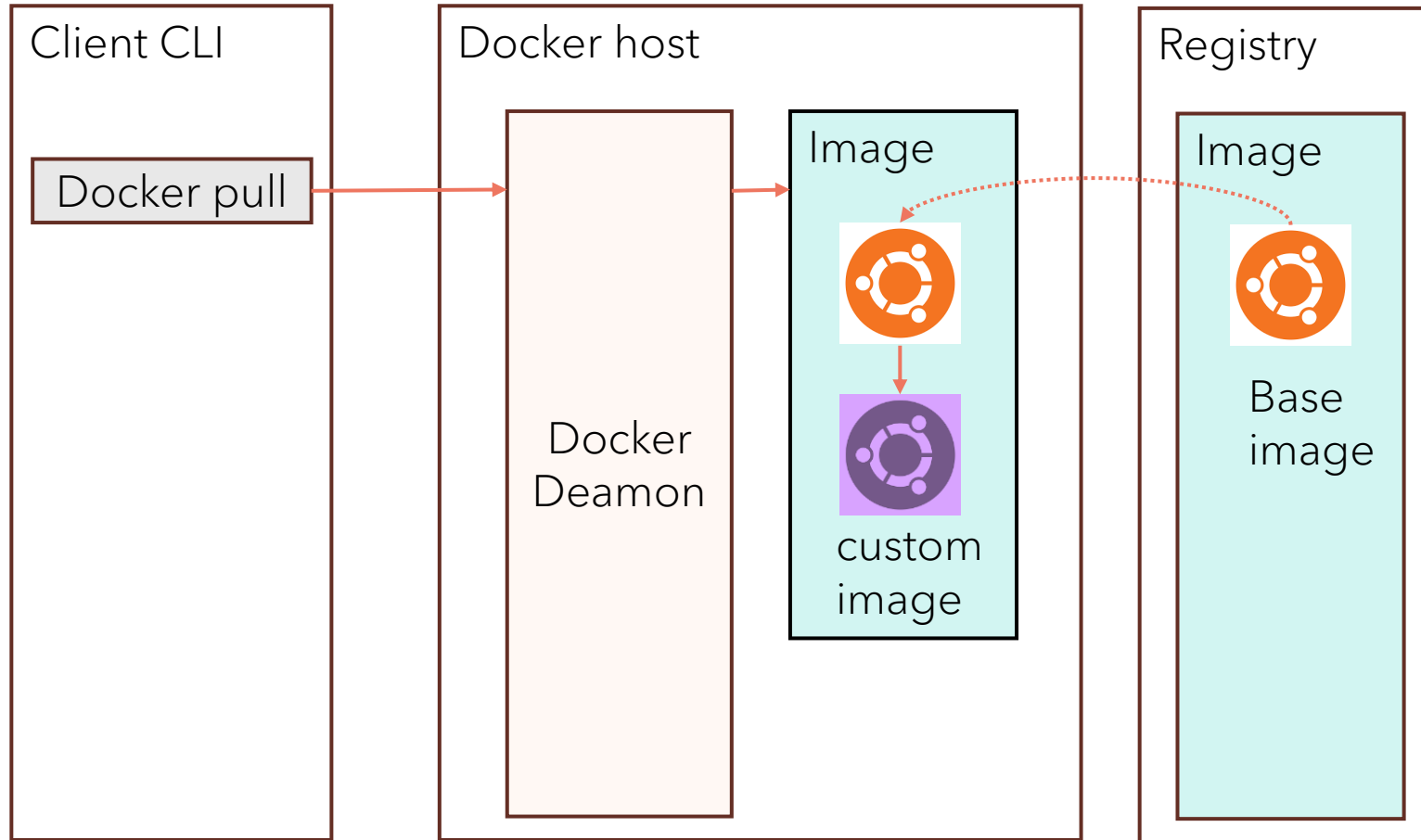
Docker run



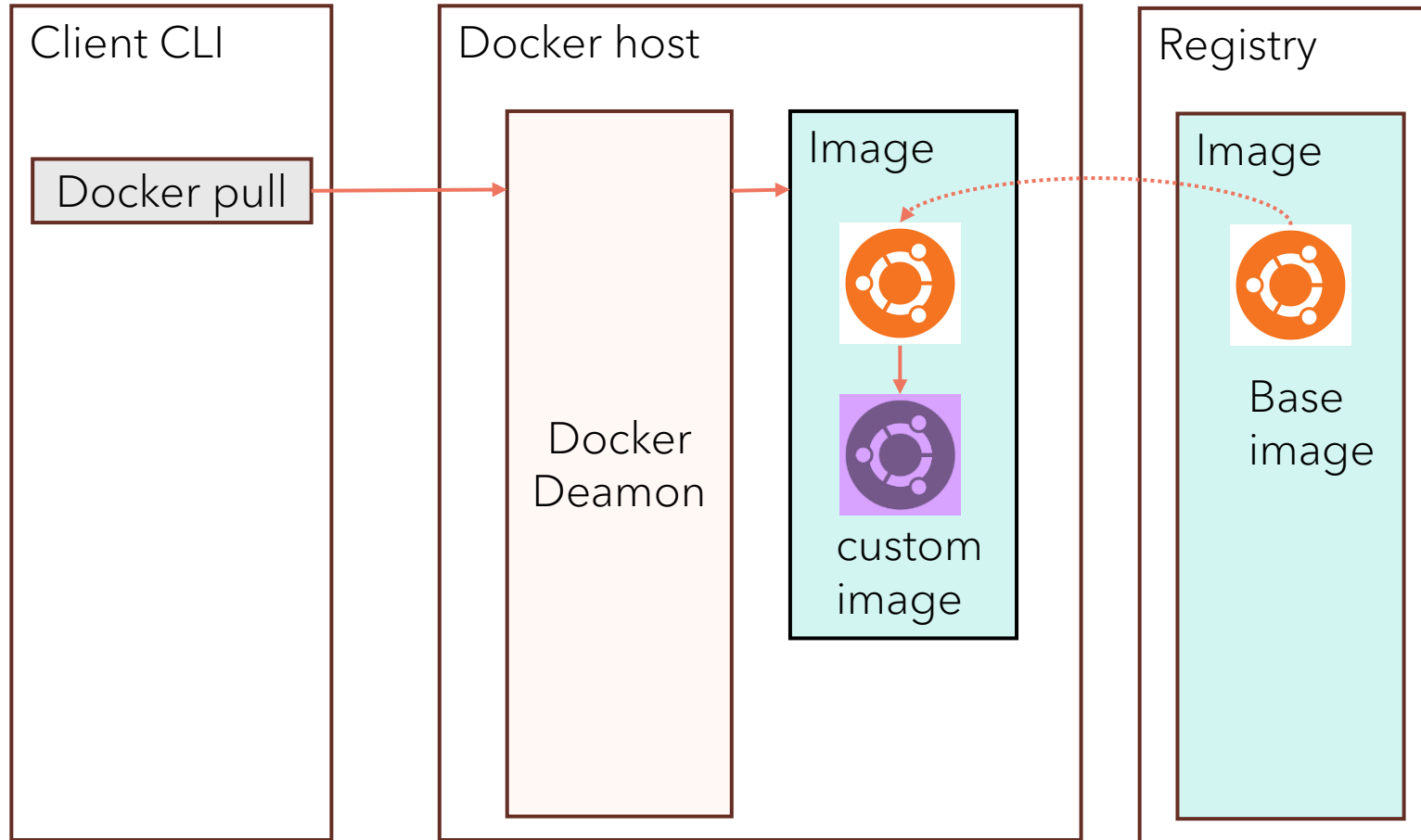
Docker pull



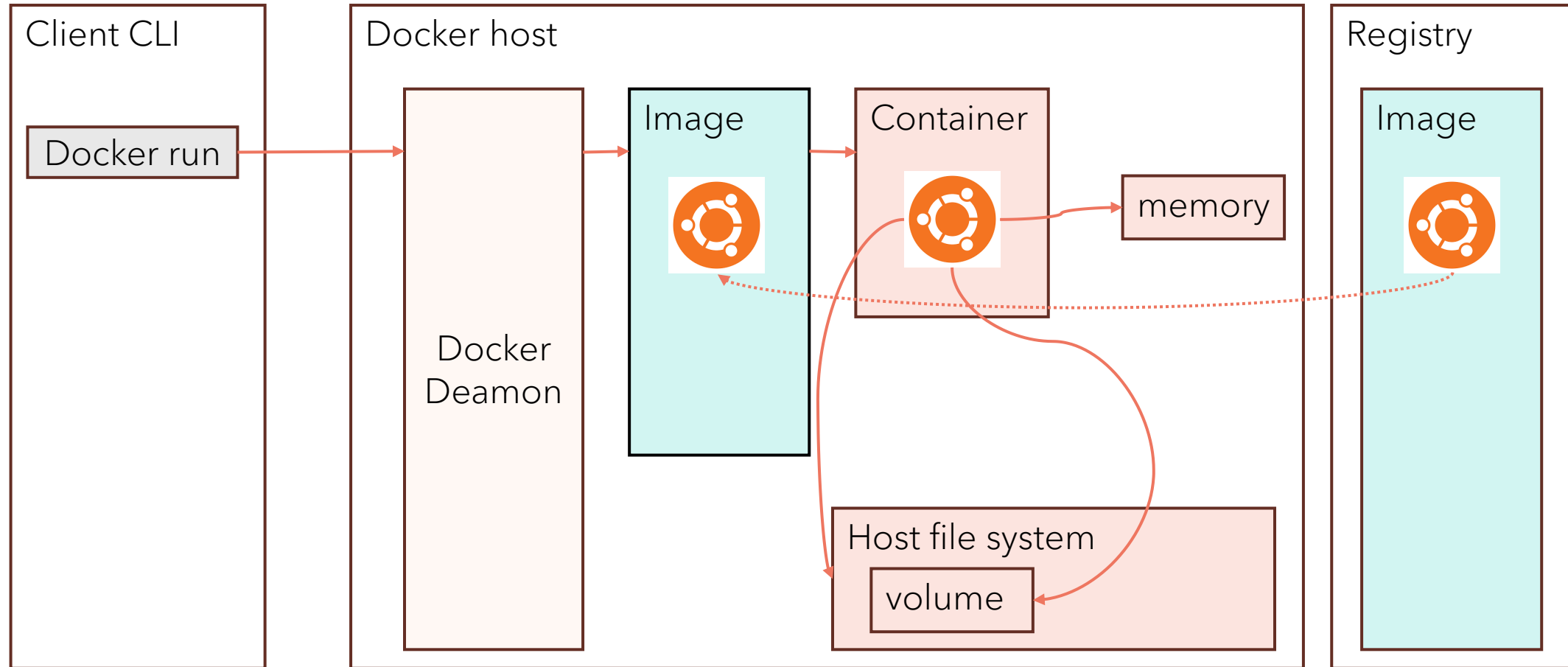
Docker build



Docker build



Docker Volume



Docker Volume

Bind Mount

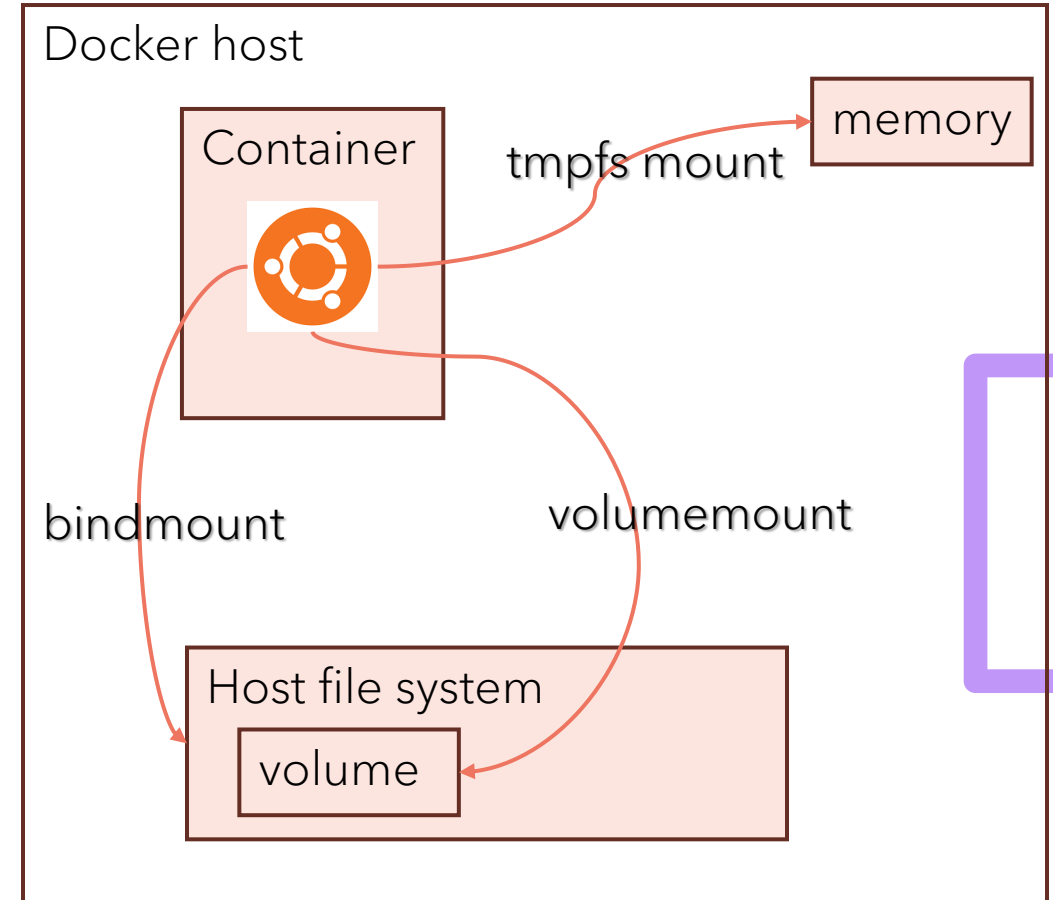
- A **bind mount** maps a specific file or directory on the host into a container.

Characteristics

- Refers to an **exact path** on your host filesystem.
- Changes in the container reflect on the host and vice versa (real-time sync).
- Useful for local development (to share source code, configuration files, etc.).

Caveats

- Docker doesn't manage the lifecycle of the host files or directories.
- Offers less portability (since it depends on your host's file paths).
- File permissions and ownership can get tricky between host and container.



Docker Volume

Volume Mount

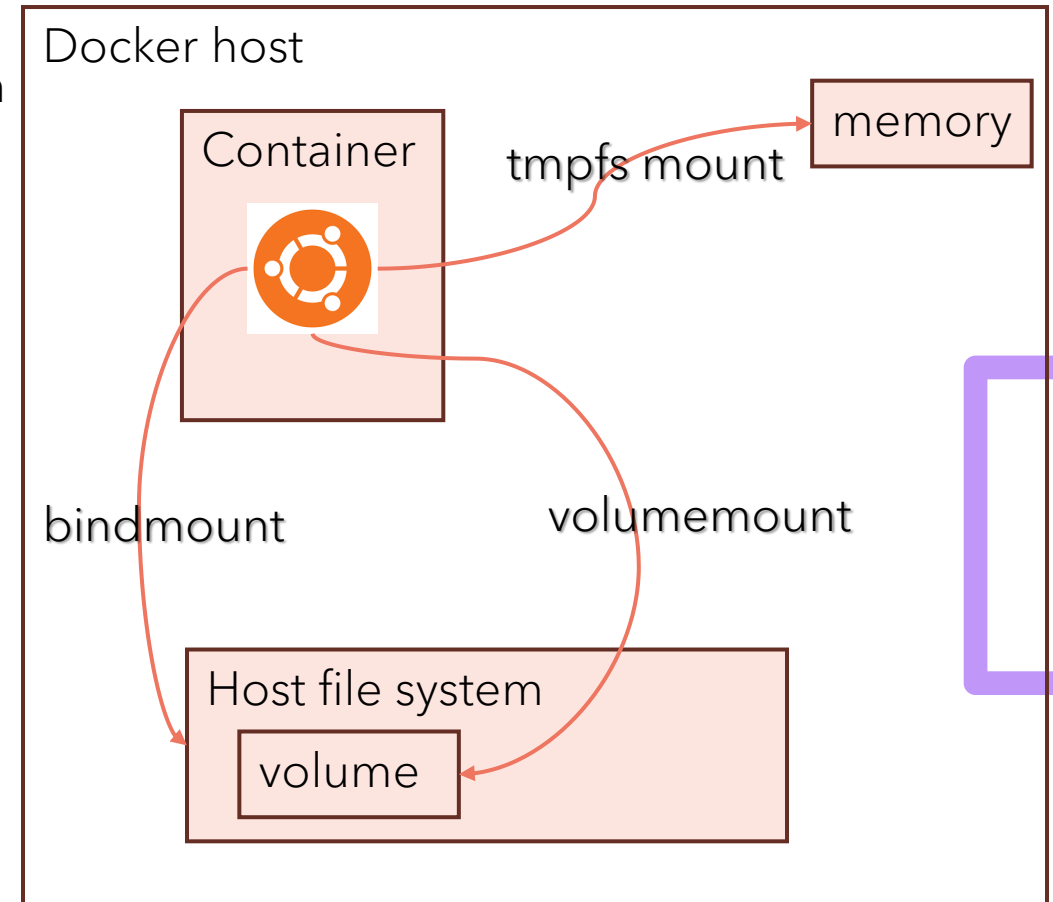
- A **volume mount** uses Docker-managed storage (on the host, but managed through Docker).

Characteristics

- Docker stores it under `/var/lib/docker/volumes/` by default.
- Volumes are **persistent, independent** of the container lifecycle.
- Best for storing data that should **survive container restarts/replaces** (databases, user uploads, etc.).
- Can be easily shared across multiple containers.

Managed by Docker

- Docker handles metadata, paths, and cleanup.
- More portable and safer than bind mounts.



Docker Volume

⚡ tmpfs Mount

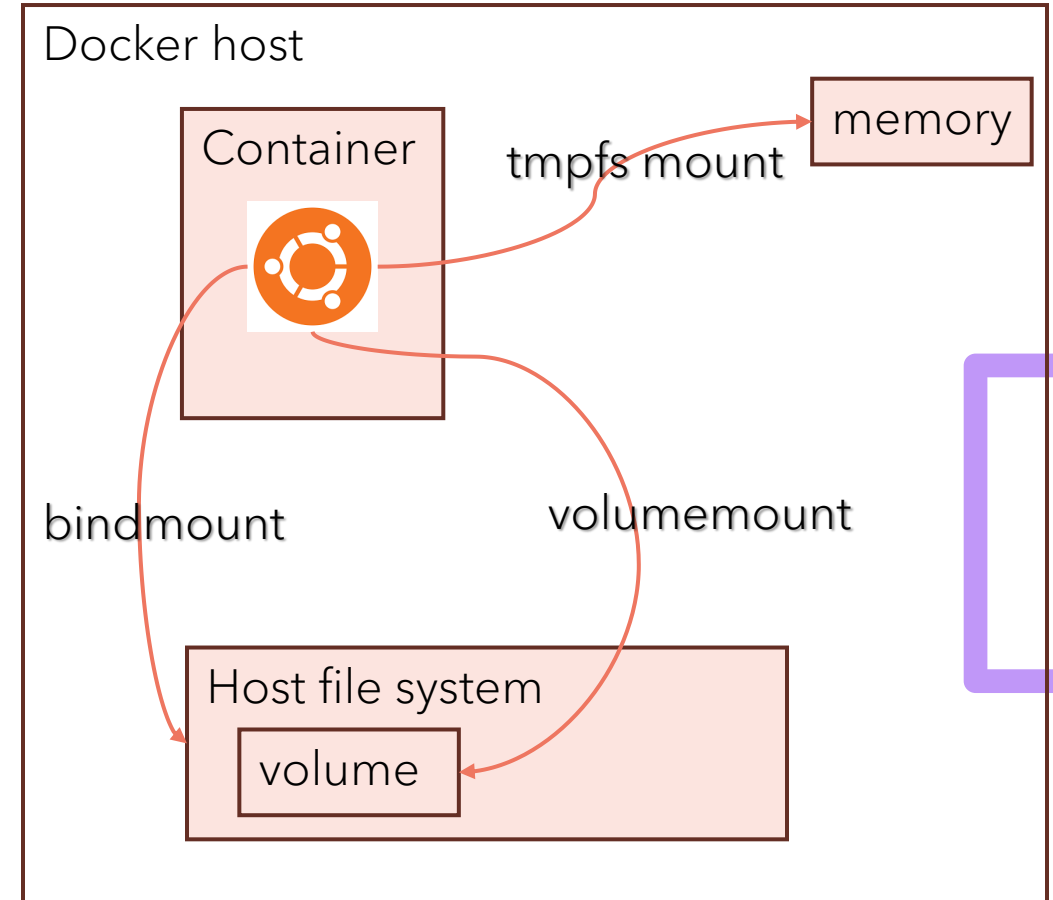
- A **tmpfs mount** stores data in memory only – not written to disk.

✅ Characteristics

- Data lives in the container's memory only.
- Automatically cleared when the container stops or restarts.
- Super fast, but volatile (non-persistent).

🧠 Use cases

- For sensitive data (credentials, tokens) that shouldn't persist.
- For caching or temporary build data.



Docker Volume

Feature	Bind Mount	Volume Mount	tmpfs Mount
Storage location	Host system path	Docker-managed storage	Host memory (RAM only)
Managed by Docker	✗ No	✓ Yes	✗ (in-memory only)
Persistence	Persists while host file exists	Persists unless removed	Lost on container stop
Performance	Host filesystem performance	Good, isolated	Fastest (memory-based)
Common usage	Dev/live file sharing	Persistent app data (DBs)	Temporary or sensitive data

Docker Network

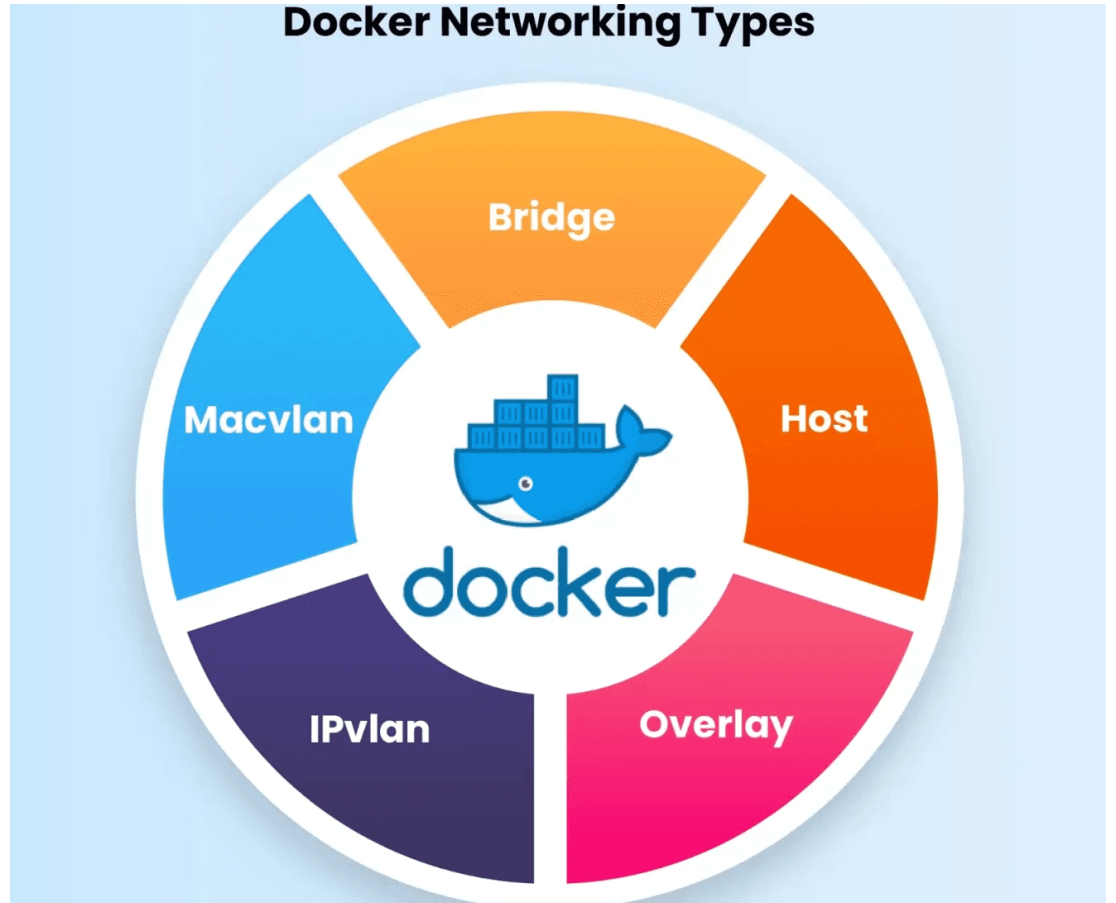
A **Docker network** is a virtual network created and managed by Docker to allow containers to communicate:

With each other (container-to-container)

With the host machine

With external networks (like the Internet)

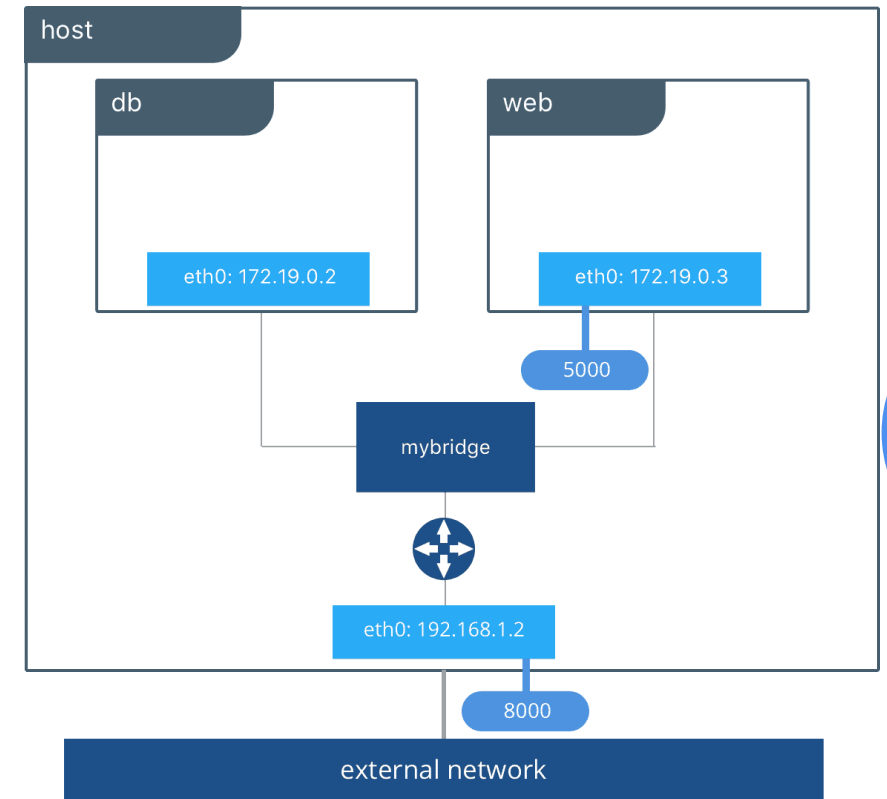
Each container can attach to one or more networks. Docker provides built-in drivers that handle different networking use cases.



Docker Network

Bridge Network

- **Use case:** Containers on the same host communicate privately.
- Each container gets its own IP within the bridge network.
- The Docker host performs **NAT (Network Address Translation)** to connect containers to the outside world.
- By default, every new container that doesn't specify a network is attached to the Docker default bridge (bridge).

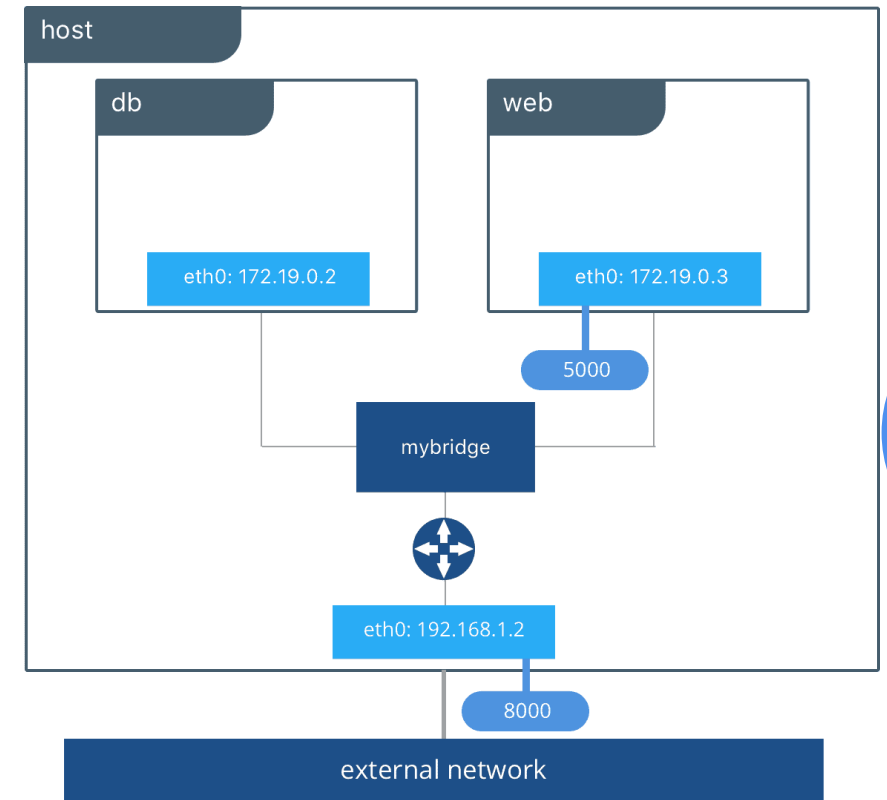


Docker Network

Bridge Network

```
.terminal
docker network create mybridge
docker run -d --name db --network mybridge nginx
docker run -d --name web -p 8000:5000 --network mybridge alpine ping web1
```

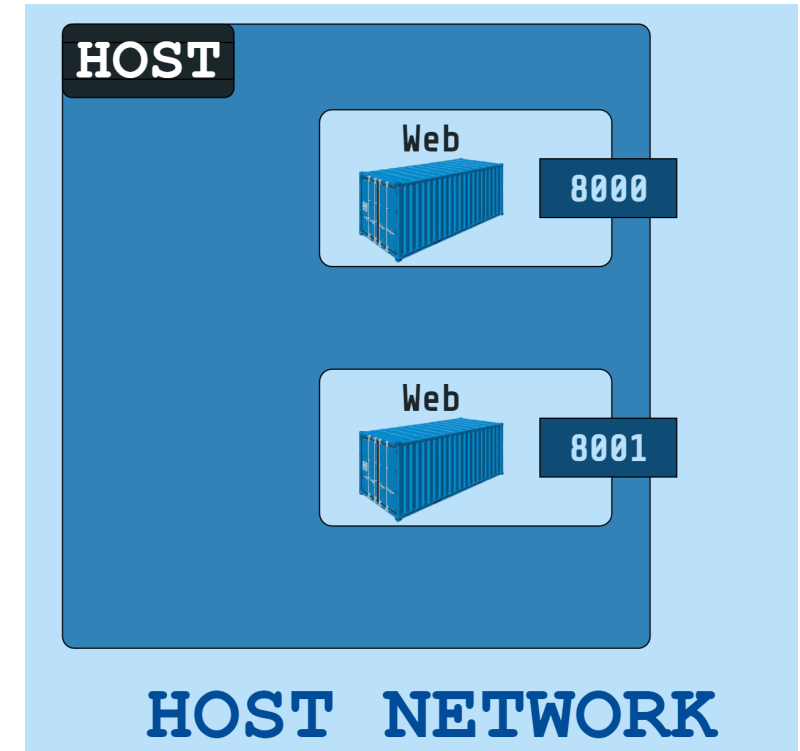
- **web** and **db** can reach each other by container name.
- **web** can be access from outside using **host-ip:8000**



Docker Network

Host Network

- **Use case:** Makes container share the host's networking stack.
- No network isolation between the container and the host.
- Container uses the **host's IP and ports** directly.
- Provides best performance but no isolation.

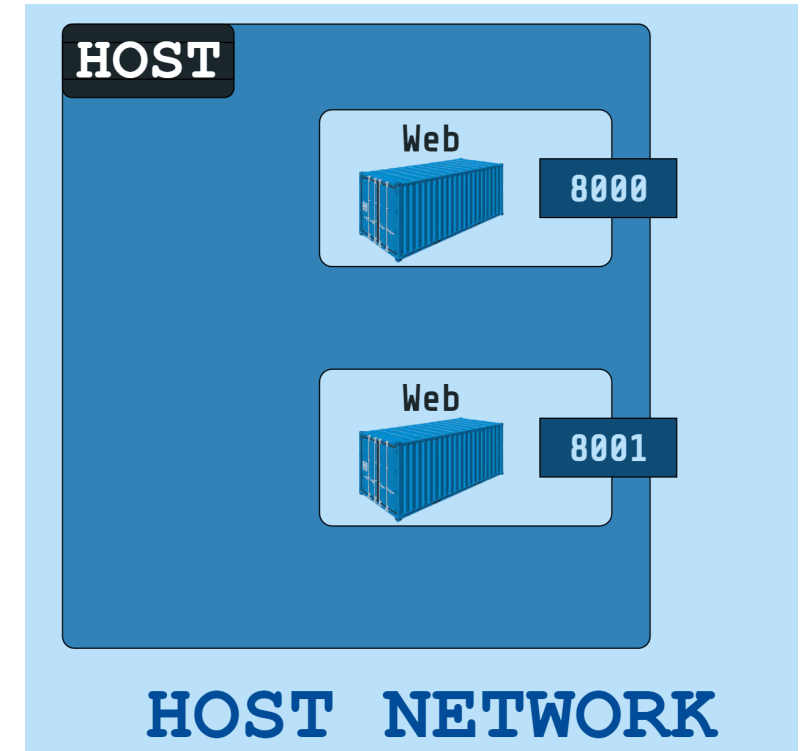


Docker Network

Host Network

```
.terminal  
docker run --network host --name web nginx
```

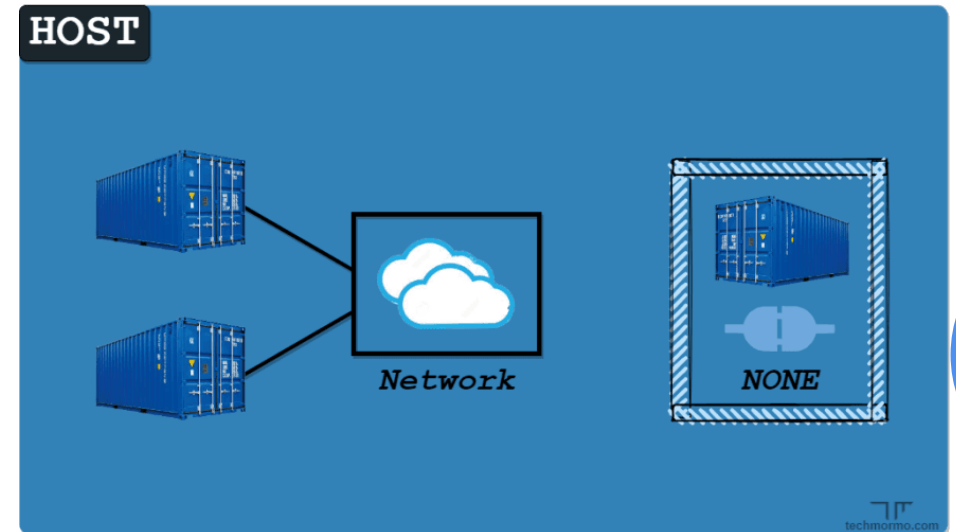
→ **web** can be access from outside using **host-ip:<container-port>**



Docker Network

None Network

- **Use case:** Completely disables networking for the container.
- No access to host, other containers, or the Internet.
- Only loopback (lo) interface available.

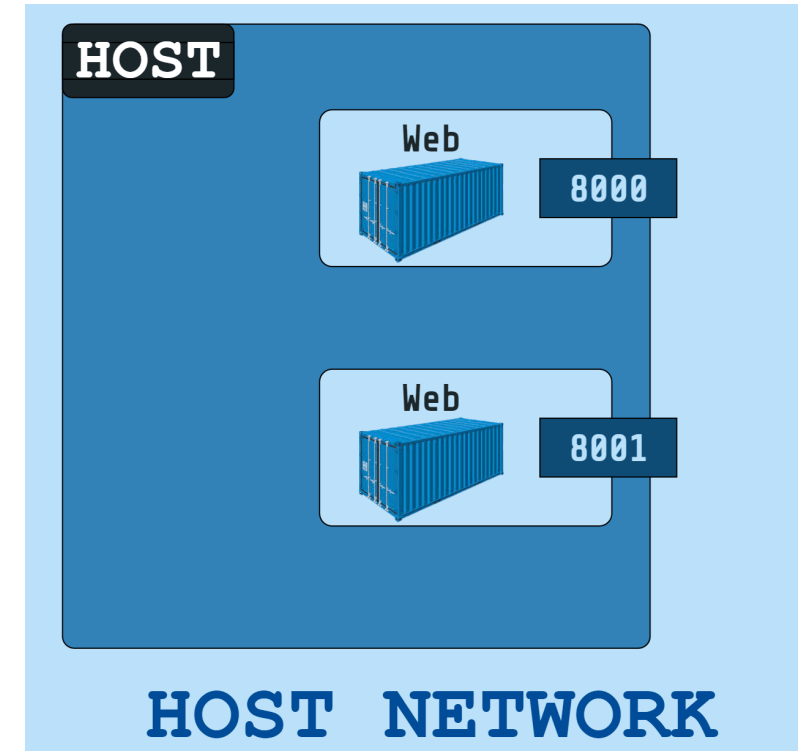


Docker Network

Host Network

```
.terminal  
docker run --network none alpine
```

→ **web** can be access from **host** using **localhost:<container-port>**



Docker CLI Cheatsheet

IMAGES

Docker images are a lightweight, standalone, executable package of software that includes everything needed to run an application: code, runtime, system tools, system libraries and settings.

Build an Image from a Dockerfile

```
docker build -t <image_name> .
```

Build an Image from a Dockerfile without the cache

```
docker build -t <image_name> . --no-cache
```

List local images

```
docker images
```

Delete an Image

```
docker rmi <image_name>
```

Remove all unused images

```
docker image prune
```

CONTAINERS

A container is a runtime instance of a docker image. A container will always run the same, regardless of the infrastructure. Containers isolate software from its environment and ensure that it works uniformly despite differences for instance between development and staging.

Create and run a container from an image, with a custom name:

```
docker run --name <container_name> <image_name>
```

Run a container with and publish a container's port(s) to the host.

```
docker run -p <host_port>:<container_port> <image_name>
```

Run a container in the background

```
docker run -d <image_name>
```

Start or stop an existing container:

```
docker start|stop <container_name> (or <container-id>)
```

Remove a stopped container:

```
docker rm <container_name>
```

Open a shell inside a running container:

```
docker exec -it <container_name> sh
```

Fetch and follow the logs of a container:

```
docker logs -f <container_name>
```



Docker cheat sheet.md



Practice Lab